

Lab Assignment-4

Computer Architecture (CS F342)

Semester-I, 2026-27

Department of Computer Science and Information Systems (CSIS)

BITS-Pilani, K K Birla Goa Campus, Goa, India.

Due time: Sept 17, 2026: 13:00

Marks: 20 (6 + 14)

/*#####

Please don't upload the assignments, template file/solution and lab. manual on GitHub or others public repository. Kindly remove them, if you have uploaded the previous assignments. It violates the BITS's Intellectual Property Rights (IPR).

*****/

Download the *CA_Lab-4_Template.zip*, and which contains 2 problems (P1 and P2), and rename it as *<CampusID>_CA_Lab-4*.

One's lab marks depend on one's performance on lab viva, even if one has no complete working code. Serious action will be taken if one has the fully working code but cannot explain it during the viva.

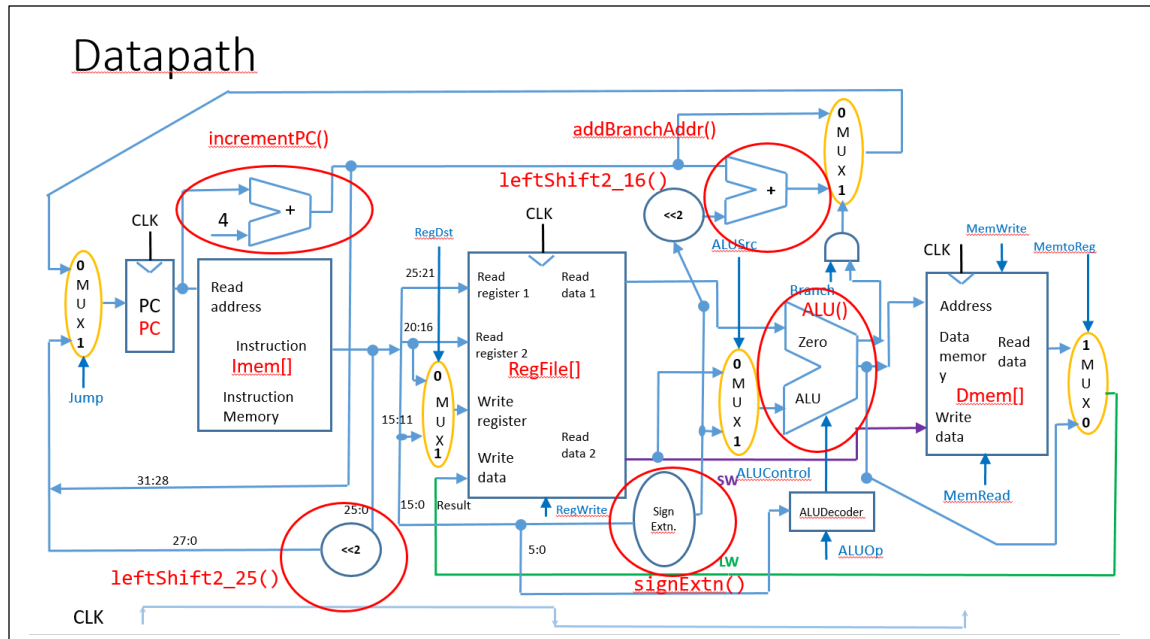
P1) See the *FiboProcessor.cpp*. No need to define other variables and to modify the other functions except the below in the program.

Consider the 32-bit MIPS instruction model (instruction format is given below) and, using CPP language, simulate the following MIPS instructions: LW, SW, ADD, ADDI, and BNE. For this, one has to define the following functions:

Marks Distribution:

Module	Marks
fetchDecodeFSM	1
lwFSM	1
swFSM	1
addFSM	1
addiFSM	1
bneFSM	1

Maintain the MIPS's single-cycle datapath while updating the different components. The datapath is given below:



STP (not MIPS instruction) instruction is used to stop the execution of the program. MIPS has 32 registers and R0 holds zero value only.

Instruction format:

R-Type	OPCODE						RS					RT					RD					SHAMT					FUNCT				
	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1
I-Type	OPCODE						RS					RT/RD					IMMEDIATE														
	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1

In the template, `load_program()` loads a program onto instruction memory to generate 7 Fibonacci numbers. In addition, it also loads the first two Fibonacci numbers at locations 252 and 248. The program is written using the above-mentioned instructions. The program can calculate the first seven Fibonacci numbers. The first two Fibonacci numbers are stored in memory locations 252 and 248, and the remaining five Fibonacci numbers can be stored in locations 244 onwards. The Big-endian format is considered to store the instruction and data. Memory is byte-addressable. Separate memory is used for the instructions and data. The memory contains 256 locations. Update the given program's variable based on the datapath. The program generates the following outputs:

```
PS D:\CSF342-CA\2023\Labs\Lab-1-ProcessorDesignWithC\CA-Lab1\sol> g++ .\FiboProcessor.cpp
PS D:\CSF342-CA\2023\Labs\Lab-1-ProcessorDesignWithC\CA-Lab1\sol> .\a.exe
The Fibonacci numbers are:
1, 1, 2, 3, 5, 8, 13,
PS D:\CSF342-CA\2023\Labs\Lab-1-ProcessorDesignWithC\CA-Lab1\sol> |
```

The memories, instruction and data, can only be accessed using read () and write (). Similarly, registers can be accessed via read() and write(). Marks can be deducted if one uses memories and register directly and if one defines the unnecessary variables.

Total marks will be awarded to your design based on Viva. If you cannot answer the question(s) asked by the evaluator, then the marks will be deducted accordingly by the evaluator, which is the final decision.

P2) Write an ASM program using MIPS ISA to check whether a number, N (take it from the user), is Happy or Unhappy. **Do not define any function.** Use the MARS simulator. Submit the following while uploading the solution.

1. HappyUnhappy.asm
2. What's the initial PC value when you run your previous asm code (assignment-2) and HappyUnhappy.asm code in the MARS simulator?
3. Take a snapshot of "count the number of instructions executed" from the MARS simulator
4. Take a snapshot of "instruction Statistics" from the MARS simulator
5. Take a snapshot of "MIPS X-Ray – Animation of MIPS Datapath" from the MARS simulator for an R-type instruction.
6. Take a snapshot of the "Branch History Table Simulator" from the MARS simulator for your code.

Marks: 5 + 1 + 2 + 2 + 2 + 2

Upload all files including output in the folder.

Instructions for uploading the results:

Make a zip file, <CampusID>_CA_Lab-4.zip which can contain the *all files*.

Do not create archives in other formats (rar, tar.gz etc). Once uploaded on Quanta, remember to **submit for grading**. **Do not leave it as a draft.**

/*#####

Note: Please don't upload the assignments, template file/solution and lab. manual on GitHub or others public repository.

It violates the BITS's Intellectual Property Rights (IPR).

*****/